

DATA & MACHINE LEARNING CLUB

Getting Started with Raspberry Pi Pico

[CircuitPython](#) • [Thonny IDE](#) • [GitHub Desktop](#) • [Starter Projects](#)

Session 1 | LUANAR, Lilongwe | 2026

Session Details	Information
Date	March 31, 2026
Location	LUANAR, Lilongwe, Malawi
Facilitators	Hobby Bwanali & Martin Chirwa
Mentors	Dexter Shepherd, Alejandra Carrero, Paula Lotte Sophia Leonie Seidler — University of Sussex, ISAID Team
Topic	Raspberry Pi Pico — Hardware Intro, CircuitPython & First Projects
Level	Absolute Beginner — No prior experience needed
Tools Required	Thonny IDE, CircuitPython firmware, GitHub Desktop

01 SESSION OBJECTIVES

By the end of this session, every member will be able to:

- Understand what the Raspberry Pi Pico is and how it differs from a regular computer
- Install CircuitPython firmware onto the Pico
- Write and run basic CircuitPython code using Thonny IDE
- Install GitHub Desktop and access club course materials and resources
- Build and run 5 starter projects using LEDs and a DHT11 temperature/humidity sensor

02 WHAT IS THE RASPBERRY PI PICO?

The Raspberry Pi Pico is a small, low-cost microcontroller board developed by the Raspberry Pi Foundation. Unlike a full Raspberry Pi computer, the Pico is designed to control electronics — read sensors, control LEDs, motors, and more. It is programmable in MicroPython and CircuitPython, making it an excellent entry point into physical computing and IoT.

Feature	Details (just sample specs)
Microcontroller	RP2040 dual-core ARM Cortex-M0+
Clock Speed	Up to 133 MHz
Memory (RAM)	264 KB on-chip SRAM

Flash Storage	2 MB on-board
GPIO Pins	26 multifunction GPIO pins
Interfaces	SPI, I2C, UART, PWM, ADC
USB	Micro-USB for power and programming
Price	Very affordable — ideal for learning
Language Support	CircuitPython, MicroPython, C/C++

03 WHAT IS CIRCUITPYTHON?

CircuitPython is a beginner-friendly version of Python created by Adafruit, designed specifically for microcontrollers like the Pico. It runs directly on the hardware, letting you write Python code to control physical components — lights, sensors, displays, and much more.

Why CircuitPython for the DML Club?

- Python syntax is clean and easy to read — ideal for new programmers
- Instant feedback: save your file and the code runs immediately on the Pico
- Huge library ecosystem for sensors, displays, communication modules
- The same data skills you learn here apply directly to data science and ML work
- Active community with thousands of tutorials and examples online

04 TOOLS SETUP GUIDE

A. Installing CircuitPython on the Pico

1. Download the latest CircuitPython .UF2 file for the Raspberry Pi Pico from circuitpython.org
2. Hold down the BOOTSEL button on your Pico, then plug it into your computer via USB — keep holding until a drive called RPI-RP2 appears
3. Drag and drop the downloaded .UF2 file onto the RPI-RP2 drive
4. The Pico will reboot automatically — a new drive called CIRCUITPY will appear
5. CircuitPython is now installed! Your code goes into the code.py file on this drive



TIP: Every time you save code.py in Thonny, the Pico automatically runs your new code.
You do NOT need to restart or re-flash the board between edits.

B. Setting Up Thonny IDE

Thonny is a beginner-friendly Python IDE that has built-in support for CircuitPython and the Raspberry Pi Pico.

6. Download Thonny from thonny.org and install it on your computer

7. Open Thonny — go to Tools > Options > Interpreter
8. Select 'CircuitPython (generic)' from the dropdown
9. Choose the correct COM port (Windows) or /dev/ttyACM0 (Linux/Mac)
10. Click OK — you should see the CircuitPython REPL appear at the bottom
11. Test it: type `print('Hello DML Club!')` and press Enter — it should print the message

⚠ TROUBLESHOOTING: If Thonny cannot find the Pico, try a different USB cable.
Some USB cables are charge-only and do not carry data. Use a data cable.

C. Installing GitHub Desktop & Accessing Course Materials

GitHub Desktop is a graphical tool that makes it easy to access, download, and sync course materials from our club repository. No command-line knowledge is needed.

12. Go to desktop.github.com and download GitHub Desktop for your operating system
13. Install and open GitHub Desktop — sign in or create a free GitHub account
14. Click 'File > Clone Repository' and paste the DML Club repository URL when provided
15. Choose a local folder on your computer to save the materials
16. Click 'Clone' — all course materials, code samples, and resources will download
17. To get updates in future sessions: click 'Fetch origin' then 'Pull' in GitHub Desktop

📌 All code samples from today's session are available in the club repository.
Facilitators will share the repository link at the start of the session.
Bookmark it and always pull the latest version before each class.

05 BASIC WIRING REFERENCE

Before we start coding, here are the wiring basics you'll use in every project today. Always build circuits with the Pico UNPLUGGED from USB.

Component	How to Connect to Pico
LED (any colour)	Long leg (+) → 330Ω resistor → GPIO pin. Short leg (-) → GND pin
Resistor (330Ω)	Always in series with an LED to limit current and protect the LED
Breadboard	Use for all connections — never solder during beginner sessions
GND (Ground)	Any pin labelled GND on the Pico pinout diagram
3.3V Power	Pin 36 (3V3 OUT) — use for sensor power supply (e.g., DHT11 VCC)
DHT11 Sensor	VCC → 3.3V, GND → GND, DATA → any GPIO (e.g., GP15)

06 STARTER PROJECTS

The following 5 projects are designed for absolute beginners. Projects 1–4 work with LEDs only. Project 5 integrates the DHT11 temperature and humidity sensor. Each project builds on the previous one — work through them in order.

PROJECT 01

Blink the Built-in LED — Hello, Pico!

This is the 'Hello World' of hardware. We'll make the LED that is built onto the Pico board itself blink on and off. No extra components needed — just the Pico and a USB cable.

What You'll Learn

- How to import CircuitPython libraries
- How to control a digital output pin
- How to create a loop that runs forever
- How to add delays in code

Components Needed

- Raspberry Pi Pico with CircuitPython installed
- USB cable connected to computer running Thonny

CircuitPython Code

```
# PROJECT 1: Blink the Built-in LED
# The Pico has a built-in LED on pin GP25
# We will make it blink on and off every second

import board          # Gives us access to the Pico's pins
import digitalio       # Lets us control digital pins (ON/OFF)
import time            # Lets us add delays

# Set up the built-in LED as an output
led = digitalio.DigitalInOut(board.LED)
led.direction = digitalio.Direction.OUTPUT

# Loop forever — the Pico will run this until unplugged
while True:
    led.value = True    # Turn LED ON
    time.sleep(1)       # Wait 1 second
    led.value = False   # Turn LED OFF
    time.sleep(1)       # Wait 1 second
```



TRY IT: Change the numbers inside `time.sleep()` to make the LED blink faster or slower.

Try `time.sleep(0.1)` for a very fast blink, or `time.sleep(3)` for a slow blink.
Save the file and watch the LED change immediately — no restart needed!

PROJECT 02

External LED Blink — Your First Circuit

Now we move from the built-in LED to an external LED connected via a breadboard. This teaches you how to wire a real circuit and control it with code — a fundamental hardware skill.

What You'll Learn

- How to wire an LED and resistor on a breadboard
- How to assign specific GPIO pins in code
- Why resistors are essential for protecting LEDs

Components Needed

- 1× LED (any colour)
- 1× 330Ω resistor
- Breadboard and jumper wires
- Pico connected to Thonny

Wiring

- LED long leg (+) → one end of 330Ω resistor
- Other end of resistor → GP15 (Pin 20 on the Pico)
- LED short leg (−) → any GND pin on the Pico

CircuitPython Code

```
# PROJECT 2: External LED Blink
# Control an LED wired to GPIO pin GP15

import board
import digitalio
import time

# Set up GP15 as a digital output
led = digitalio.DigitalInOut(board.GP15)
led.direction = digitalio.Direction.OUTPUT

print('LED Blink Project running...')

while True:
```

```
led.value = True      # Turn external LED ON
print('LED is ON')    # Print to Thonny console
time.sleep(0.5)       # Wait half a second

led.value = False     # Turn external LED OFF
print('LED is OFF')   # Print to Thonny console
time.sleep(0.5)       # Wait half a second
```

- 💡 **TRY IT:** Add a second LED on GP16 and make the two LEDs alternate — one ON while the other is OFF. This is the basis of a traffic light system!
- 🔴 **Notice:** We added `print()` statements — these appear in Thonny's console and are very useful for debugging and understanding what your code is doing.

PROJECT 03

LED Traffic Light — Sequencing & Timing

We now control three LEDs in sequence to simulate a traffic light — Red, Yellow, and Green. This introduces multi-output control, timing logic, and real-world system thinking.

What You'll Learn

- How to manage multiple output pins at the same time
- How to sequence events using timing
- How to turn multiple LEDs OFF and then selectively turn one ON
- Thinking about systems: input → process → output

Components Needed

- 3× LEDs (Red, Yellow/Orange, Green)
- 3× 330Ω resistors
- Breadboard and jumper wires

Wiring

- Red LED → 330Ω resistor → GP13
- Yellow LED → 330Ω resistor → GP14
- Green LED → 330Ω resistor → GP15
- All LED short legs (–) → GND

CircuitPython Code

```
# PROJECT 3: LED Traffic Light
# Simulate a traffic light with Red, Yellow, and Green LEDs

import board
import digitalio
import time

# Set up all three LED pins as outputs
red    = digitalio.DigitalInOut(board.GP13)
yellow = digitalio.DigitalInOut(board.GP14)
green  = digitalio.DigitalInOut(board.GP15)

red.direction    = digitalio.Direction.OUTPUT
yellow.direction = digitalio.Direction.OUTPUT
green.direction  = digitalio.Direction.OUTPUT

def all_off():
    """Helper function to turn all LEDs off at once"""
    red.value = False
    yellow.value = False
    green.value = False

print('Traffic Light starting...')

while True:
    # RED phase — Stop
    all_off()
    red.value = True
    print('RED    - Stop')
    time.sleep(3)

    # YELLOW phase — Get Ready
    all_off()
    yellow.value = True
    print('YELLOW - Get Ready')
    time.sleep(1)

    # GREEN phase — Go
    all_off()
    green.value = True
    print('GREEN  - Go')
    time.sleep(3)
```



TRY IT: Add a pedestrian crossing! When the light is RED for cars, make a 4th LED (white or blue) light up to show it is safe for pedestrians to cross.



NOTICE the all_off() function — writing reusable functions makes your code cleaner and easier to read. This is a key habit in professional coding.

PROJECT 04

LED Brightness Control — PWM Fading

In projects 1–3, LEDs were either fully ON or fully OFF (digital control). Now we use PWM (Pulse Width Modulation) to smoothly fade an LED in and out — like breathing. This is the foundation for motor speed control, dimmer switches, and audio output.

What You'll Learn

- What PWM (Pulse Width Modulation) is and how it works
- How to use the pwmio library in CircuitPython
- How to create smooth animations using loops and math
- The difference between digital output and PWM output

Components Needed

- 1× LED (any colour)
- 1× 330Ω resistor
- Breadboard and jumper wires

Wiring

- LED long leg (+) → 330Ω resistor → GP15
- LED short leg (–) → GND

CircuitPython Code

```
# PROJECT 4: LED Fade (PWM Brightness Control)
# Smoothly fade an LED in and out using PWM

import board
import pwmio    # PWM (Pulse Width Modulation) library
import time

# Set up GP15 as a PWM output at 5000 Hz frequency
led = pwmio.PWMOut(board.GP15, frequency=5000, duty_cycle=0)

# PWM duty_cycle ranges from 0 (fully off) to 65535 (fully on)
MAX_BRIGHTNESS = 65535
STEP = 1000      # How much to change brightness each step
DELAY = 0.01     # Delay between each step (seconds)

print('LED Fade Project running...')

while True:
    # Fade IN – brightness increases from 0 to maximum
    print('Fading IN...')
    for brightness in range(0, MAX_BRIGHTNESS, STEP):
        led.duty_cycle = brightness
        time.sleep(DELAY)
```



```
# Fade OUT — brightness decreases from maximum to 0
print('Fading OUT...')
for brightness in range(MAX_BRIGHTNESS, 0, -STEP):
    led.duty_cycle = brightness
    time.sleep(DELAY)

time.sleep(0.3) # Short pause at the bottom before repeating
```

 TRY IT: Change STEP to 500 for a smoother, slower fade.

Change STEP to 5000 for a faster, choppy fade.

Try connecting 3 LEDs on different pins and fading them at different speeds to create a 'breathing light' effect like a laptop standby indicator!

PROJECT 05 DHT11 Temperature & Humidity — Your First Sensor

We now introduce the DHT11 — a digital temperature and humidity sensor. This is directly linked to the DML Club's mission: collecting environmental data. We will read data from the sensor and display it in real time using Thonny, then use LEDs to give a visual alert if temperature exceeds a threshold.

What You'll Learn

- How to wire and communicate with the DHT11 sensor
- How to read real-world data (temperature, humidity) in code
- How to use conditional logic (if/else) based on sensor readings
- How to combine sensor data with LED output for a simple alert system
- The basics of data collection — the first step in any ML pipeline

Components Needed

- 1× DHT11 temperature and humidity sensor
- 1× 10kΩ pull-up resistor (connect between DATA and VCC pins of DHT11)
- 2× LEDs (Green and Red)
- 2× 330Ω resistors (for LEDs)
- Breadboard and jumper wires

Wiring

- DHT11 VCC → Pico 3.3V (Pin 36)
- DHT11 GND → Pico GND
- DHT11 DATA → GP15 (also connect a 10kΩ resistor between DATA and VCC)
- Green LED → 330Ω resistor → GP13 (Normal temperature indicator)
- Red LED → 330Ω resistor → GP14 (High temperature warning)

Step 1 — Install the DHT Library

CircuitPython needs a library file for the DHT11 sensor. Download the Adafruit CircuitPython DHT library bundle from circuitpython.org/libraries. Copy `adafruit_dht.mpy` into the `/lib` folder on your CIRCUITPY drive.

CircuitPython Code — Part A: Read & Display Sensor Data

```
# PROJECT 5A: Read Temperature and Humidity from DHT11
# Display the readings in Thonny and on LEDs

import board
import digitalio
import adafruit_dht # DHT11 / DHT22 sensor library
import time

# Set up DHT11 sensor on GP15
dht_sensor = adafruit_dht.DHT11(board.GP15)

# Set up indicator LEDs
green_led = digitalio.DigitalInOut(board.GP13)
red_led = digitalio.DigitalInOut(board.GP14)
green_led.direction = digitalio.Direction.OUTPUT
red_led.direction = digitalio.Direction.OUTPUT

TEMP_THRESHOLD = 30 # Degrees Celsius — adjust for your environment

print('DML Club — Temperature & Humidity Monitor')
print('=====')

while True:
    try:
        # Read from the sensor
        temperature = dht_sensor.temperature # in degrees Celsius
        humidity = dht_sensor.humidity # in percent (%)

        # Display the readings
        print(f'Temperature: {temperature:.1f} °C')
        print(f'Humidity: {humidity:.1f} %')
        print('---')

        # LED alert: Red LED if temperature exceeds threshold
        if temperature >= TEMP_THRESHOLD:
            red_led.value = True # WARNING: temperature high
            green_led.value = False
            print(f'⚠ WARNING: Temperature above {TEMP_THRESHOLD}°C!')
        else:
            green_led.value = True # Temperature is normal
            red_led.value = False

    except RuntimeError as e:
        # The DHT11 sometimes fails to read — just try again
```

```
print('Sensor read error – retrying...')

time.sleep(2)    # DHT11 needs at least 2 seconds between readings
```

CircuitPython Code — Part B: Simple Data Logger

This extends Project 5A by logging readings with a timestamp counter, simulating the data collection pipelines used in real ML and data science projects.

```
# PROJECT 5B: Simple Data Logger – Save readings with a record number
# This simulates what a real environmental data collection system does

import board
import adafruit_dht
import time

dht_sensor = adafruit_dht.DHT11(board.GP15)

reading_number = 1    # Counter for each data record

print('DML Club – Data Logger')
print('#   | Time (s) | Temp (°C) | Humidity (%)')
print('-' * 45)

start_time = time.monotonic()    # Record the start time

while True:
    try:
        temperature = dht_sensor.temperature
        humidity     = dht_sensor.humidity

        # Calculate elapsed time since logging started
        elapsed = time.monotonic() - start_time

        # Print a formatted data row
        print(f'{reading_number:<3}| {elapsed:>7.1f}s | {temperature:>9.1f} | {humidity:>12.1f}')

        reading_number += 1

    except RuntimeError:
        print('Read error – skipping this reading')

    time.sleep(2)
```



THIS IS DATA SCIENCE IN ACTION! The output from Project 5B is structured data.

If you copy it into a spreadsheet or CSV file, you have a real dataset that you could visualise in Python, Excel, or use to train a machine learning model.

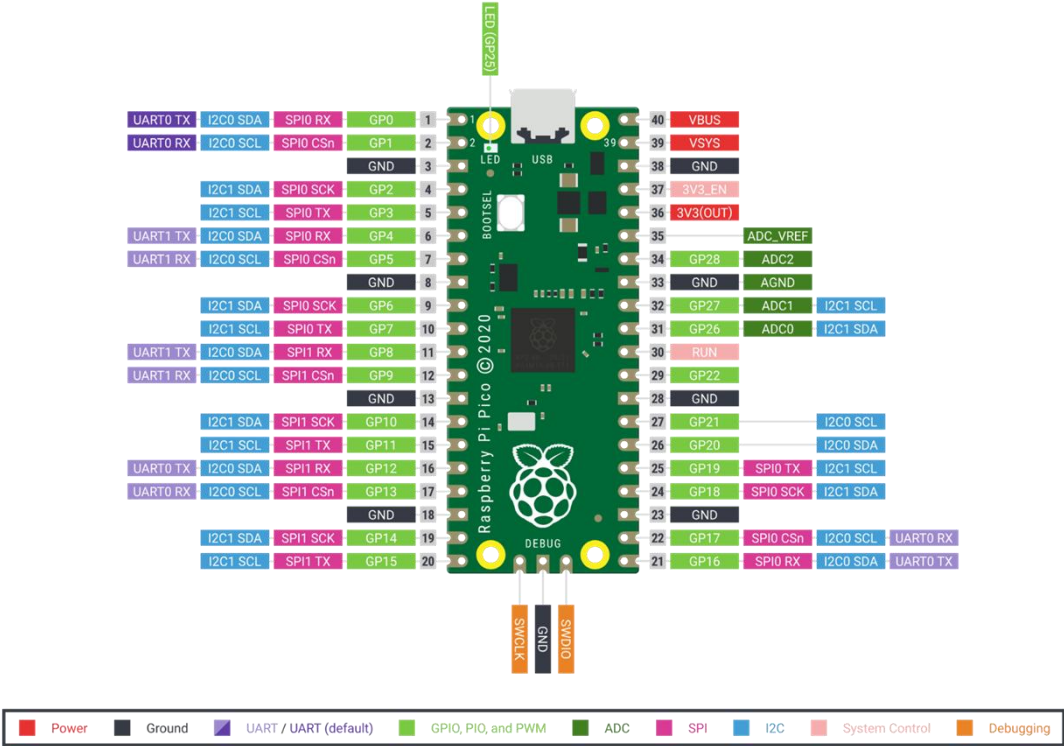
This is exactly how environmental monitoring and smart agriculture systems work!



In future sessions we will learn how to save this data directly to a file

and upload it for analysis — completing the full data pipeline.

07 QUICK REFERENCE — CircuitPython Essentials



Code Snippet	What It Does
import board	Access the Pico's GPIO pins by name (GP0, GP1 ... GP28, LED)
import digitalio	Control digital pins: ON (True) or OFF (False)
import pwmio	Control PWM pins for variable brightness / speed
import adafruit_dht	Read temperature and humidity from DHT11 / DHT22 sensors
import time	Add delays using time.sleep() and measure elapsed time
while True:	Infinite loop — runs forever until the Pico is unplugged or reset
time.sleep(1)	Pause the code for 1 second (use decimals for shorter pauses)
print('hello')	Send text to the Thonny console — great for debugging
led.value = True/False	Turn a digital output pin ON (True) or OFF (False)
led.duty_cycle = 32767	Set PWM brightness to ~50% (0 = off, 65535 = full on)
try: ... except:	Handle errors gracefully — essential when reading sensors

08 WHAT'S COMING NEXT

Now that you can control outputs and read sensors, the following sessions will build directly on these skills:

Upcoming Session	Topics Covered
Session 2 — Inputs & Displays	Push buttons, potentiometers, OLED displays, user interaction
Session 3 — Data Collection Systems	Saving data to CSV files, multiple sensors, structured datasets
Session 4 — Data Analysis	Loading CSV data in Python, pandas, matplotlib visualisations
Session 5 — Introduction to ML	Training a simple model on data you collected with the Pico
Session 6 — Real-World Project	Full pipeline: sensor → data → model → alert/decision system

09 RESOURCES & FURTHER READING

- CircuitPython Documentation: docs.circuitpython.org
- Adafruit CircuitPython Libraries: circuitpython.org/libraries
- Thonny IDE: thonny.org
- GitHub Desktop: desktop.github.com
- Raspberry Pi Pico Pinout Diagram: raspberrypi.com/documentation/microcontrollers
- DML Club Repository: see facilitator for the latest GitHub URL



🔴 DML CLUB CONTACTS

Hobby Bwanali: hobbybwanali@gmail.com | +265 885 134 501

Martin Chirwa: martinchirwa508@gmail.com | +265 997 253 881

Location: LUANAR, Lilongwe, Malawi

From Data to Decisions — Building Tomorrow's Innovators Today

Data & Machine Learning Club © 2026